Quantum HyperGraph (QHG)

A First-Class Data Model for Rule-Bearing Knowledge

true

21 April 2026

Abstract

Knowledge graphs have become the default structural backbone for reasoning over large text corpora, from enterprise knowledge bases to agent memory. Existing backbones — property graphs, RDF, labelled-property graphs — share a primitive: an edge connects *exactly two* nodes. This primitive is well suited to entity-relation data but fails to express the joint constraints that define a regulation, a contract clause, or a multi-premise inference: a single obligation routinely depends on three or more rules holding simultaneously, with exceptions that further modify the joint state.

This paper introduces **Quantum HyperGraph (QHG)**, the data model underneath QGI's QAG engine [QGI-TR-2026-01]. QHG is a hypergraph — edges connect arbitrary numbers of nodes — extended with a typed, immutable, versioned semantic that makes *conflict* and *dependency* first-class edge types. We describe the data model, its query interface, its versioning discipline, and its integration with the Hilbert-space signal layer that sits on top of it. We contrast QHG with property graphs, RDF, and the hypergraph approaches that have recently appeared in the retrieval-augmented literature. Our claim is not that hypergraphs are new — they are not — but that *conflict-as-first-class-edge* plus *immutable versioning* plus *signed-interference annotation* is a combination that has not, to our knowledge, been deployed at production scale, and that this combination is the right substrate for reasoning over rule-bearing content.

Contents

1 Why a hypergraph, and why versioned **2**

2 The QHG data model **3**
2.1 Nodes . . . 3
2.2 Hyperedges . . . 3
2.3 Versioning . . . 4

3 Query interface **4**
3.1 Node-query by role . . . 4
3.2 Edge-query by type and degree . . . 4
3.3 Neighbourhood-query with Hilbert-space weights . . . 4

4 The Hilbert-space signal layer **4**

5 Comparison with existing data models **5**
5.1 Property graphs (Neo4j, Memgraph, TigerGraph) . . . 5
5.2 RDF / triple stores . . . 5
5.3 Hypergraph databases (HyperGraphDB, Grakn / TypeDB) . . . 5
5.4 Recent hypergraph-in-RAG work . . . 6

6 Storage and implementation notes **6**

7 Limitations **6**

8 Related work **7**

9 Competing interests **7**

10 References **7**

11 Cite this as **7**

12 Version history **8**

Preprint — v1.0, 21 April 2026. Not peer-reviewed. Authored by Dr. Sam Sammane, CTO and Founder, Quantum General Intelligence, Inc. (`sam@qgi.dev`). Companion to [QGI-TR-2026-01]. Comments welcome at `research@qgi.dev`.

1 Why a hypergraph, and why versioned

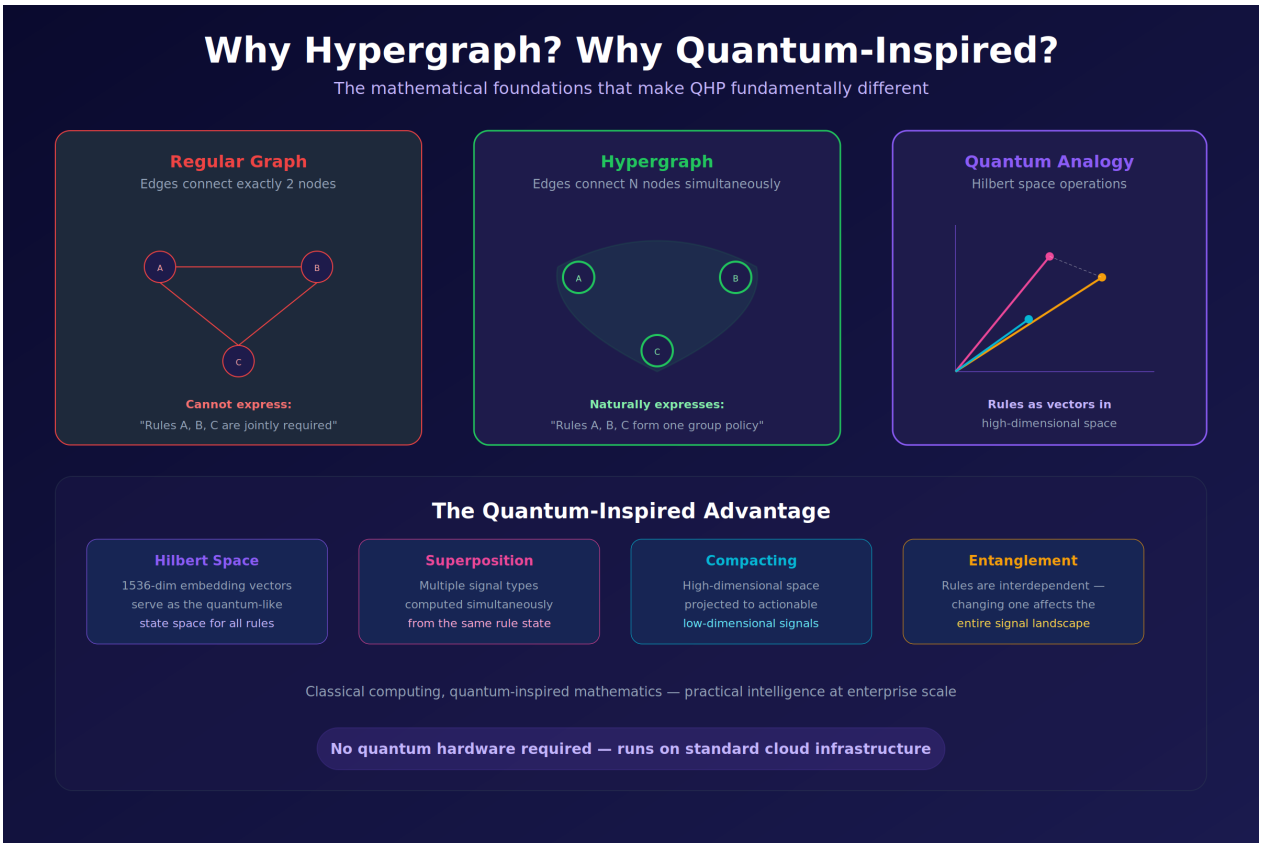

Figure 1. A hypergraph generalises a graph by letting a single edge connect N nodes at once.

A standard graph has edges that connect exactly two nodes. That primitive is expressive enough for A-is-related-to-B statements and is the right model for friendship networks, call graphs, and citation networks. It is the wrong model for a class of content that every regulated enterprise and every long-running agent accumulates: *rule sets* in which a single obligation or inference depends on several propositions holding simultaneously.

Consider a compliance example: "For activity X, conducted by actor Y, in jurisdiction Z, subject to exception E, the reporting rule is R." There is no pairwise edge that captures this. There is a simultaneous constraint over $\{X, Y, Z, E, R\}$ that holds jointly or not at all. Modelling it as a chain of pairwise edges loses the joint semantics: a query asking "when does R apply?" returns partial answers that a reasoner must reconstruct. A hypergraph avoids the reconstruction.

The second choice — *versioned and immutable* — follows from the audit requirement. Regulated systems routinely need to answer "what was the rule on date t?" and "which rules led to this decision?" Mutable graphs lose that property: rules are updated in place, and the graph state at t can only be reconstructed by replaying a log. An immutable versioned hypergraph makes every intermediate state addressable and comparable; the graph state at t is always available as a first-class object.

QHG is the combination: a hypergraph whose edges carry typed semantics (dependency, conflict, activation) and whose state is immutable and versioned.

2 The QHG data model

2.1 Nodes

QHG has four node types, corresponding to the four atomic units of rule-bearing text:

Type	Meaning
`rule`	A structured conditional statement (trigger, condition, action, exception).
`condition`	A predicate extracted from a rule's body.
`action`	The obligation or outcome the rule imposes.
`entity`	A real-world referent (actor, jurisdiction, time window, artefact).

Nodes are content-addressed: a `rule` node's identity is the hash of its QNR2 AST, so two identical rules extracted from different sources produce the same node.

2.2 Hyperedges

Hyperedges connect arbitrary numbers of nodes. They carry a type from a closed taxonomy:

Type	Semantic
`dependency`	Edge connects a rule that depends on one or more other rules for activation, scope, or precedence.
`conflict`	Edge connects two or more rules that produce contradictory outcomes on a shared trigger. Carries a *severity* and a *signed interference value*.
`overlap`	Edge connects two or more rules whose conditions intersect without necessarily contradicting.
`activation`	Edge connects a rule to the conditions and entities that activate it.
`derivation`	Edge connects a rule to its source span in the ingested document.
`supersession`	Edge connects a rule version to the version it replaces.

Six types cover the full reasoning surface QAG exposes. Extensions are treated as *new* types with their own semantics, versioned, not as variations on existing types.

2.3 Versioning

A QHG is immutable at the snapshot level. A single snapshot is a content-addressed, acyclic commitment: the hash of its node set, edge set, and type annotations. Updates produce a new snapshot. A *supersession* edge links the new snapshot's version of a rule to its predecessor.

Three concrete properties follow.

Replay. Any past answer produced by the QAG engine can be replayed against the snapshot it was produced against. This is the property that turns "what rule was applied on date t?" from a log-grep into a snapshot lookup.

Diff. Two snapshots admit a well-defined diff: nodes added, nodes removed, edges added, edges removed, edge types changed. Diffs are the atomic units of change-review.

Branching. A snapshot can be branched — a hypothetical rule set, an upcoming regulatory change, a what-if analysis — without touching the production graph. Branches merge back into mainline under ordinary diff review.

3 Query interface

QHG queries are structural. Three primitive patterns cover the majority of real deployments.

3.1 Node-query by role

Return all rule nodes whose `role` field matches a given value, or whose structured AST matches a pattern. Example: "every rule whose `exception` mentions jurisdiction Z". Runs in $O(|E_{\text{exception}}|)$ given an index on the `exception` role.

3.2 Edge-query by type and degree

Return all hyperedges of type t whose cardinality matches a predicate. Example: "every `conflict` edge of cardinality ≥ 3 — three-way conflicts, which require escalation". Runs in $O(|E_{\text{conflict}}|)$.

3.3 Neighbourhood-query with Hilbert-space weights

Starting from a set of seed nodes, expand neighbourhoods along a chosen edge type up to depth d, with the results re-ranked by the HSC signal appropriate to the edge type. Example: "every rule connected to rule R by a `conflict` edge, re-ranked by the interference signal's absolute magnitude". Runs in $O(d \cdot \text{avg-degree})$ with HSC re-ranking a $O(kd)$ operation in the embedding dimension.

The three primitives compose. A full compliance query — "every active rule in jurisdiction Z that is in conflict with company policy P, with the strongest conflict listed first" — is expressible as a single composed expression over the three primitives.

4 The Hilbert-space signal layer

QHG is the graph; HSC is the signal layer. Each HSC signal is a closed-form projection of the joint state of a subset of QHG nodes onto a named subspace of the underlying Hilbert space. Six of the seven HSC signals described in [QGI-TR-2026-01] annotate QHG edges directly:

HSC signal	QHG element annotated
Relevance	Rule nodes, ranked with respect to a query centroid
Conflict	Pairs of rule nodes joined by a `conflict` edge; the edge carries the signed interference value
Overlap	Pairs of rule nodes joined by an `overlap` edge; the edge carries the intersection percentage
Redundancy	Clusters of rule nodes; the cluster is a hyperedge of degree ≥ 2
Coverage	A map over entity nodes; the map is an annotation of the entity set
Topology	A set of graph-structural annotations on the QHG itself (components, cycles, reachability)

A signal is a *read* of the graph, not a *write*: the QHG itself is immutable, and the signal layer produces its annotations on demand from the current snapshot. Caching is performed at the snapshot level; annotations are content-addressed by (snapshot hash, signal type).

This separation is what gives QAG its reproducibility property. Given a snapshot hash and a signal type, the annotation value is a deterministic function of the two; two runs one month apart, on the same snapshot, must produce the same annotation.

5 Comparison with existing data models

5.1 Property graphs (Neo4j, Memgraph, TigerGraph)

Property graphs use pairwise edges with key-value properties. Joint constraints are expressible only by reification: a joint constraint becomes a node, and each participant receives a pairwise edge to the reified node. The reification works but inflates the graph, obscures the underlying semantics, and requires schema discipline that is difficult to maintain at scale.

Property graphs have no first-class notion of immutability or versioning; mainline property-graph backends are mutable. A versioned deployment requires an application-level layer that serialises snapshots, which is feasible but uncommon.

5.2 RDF / triple stores

RDF is the standards-based triple-store formalism behind the semantic web. A triple $\langle s, p, o \rangle$ is, again, a pairwise relation. Joint constraints are expressible through reification via `rdf:Statement`, but reification is so cumbersome in practice that few production deployments use it. Named graphs provide partial snapshotting but do not give the diff or branch semantics of an immutable snapshot model.

5.3 Hypergraph databases (HyperGraphDB, Grakn / TypeDB)

HyperGraphDB [@hypergraphdb] and TypeDB provide general-purpose hyperedge support. Both are mutable in their mainline deployments and none provide the typed *conflict*, *dependency*, and *supersession* semantics, nor the signed-interference annotation layer. They are viable substrates for QHG in principle; QGI's production deployment uses a custom backend because the indexing patterns (signal-re-ranked neighbourhood queries, snapshot-level content addressing) are not standard

hypergraph-database operations.

5.4 Recent hypergraph-in-RAG work

A recent line of retrieval-augmented work has begun to use hypergraphs over LLM-extracted facts (HyperGraphRAG [@luo2024hypergraphrag], others). These systems share the hyperedge primitive with QHG but differ in three important respects:

1. The graph is *built by an LLM at ingestion*. QNR2 is deterministic; the hypergraph is reproducible.
2. *Conflict* is not a first-class edge type. These systems retrieve by similarity and merge contradictions.
3. *Immutability and versioning* are not modelled; the graph evolves in place.

The difference matters in production. QHG's immutability and conflict-as-first-class-edge are the properties that make the graph auditable, not merely queryable.

6 Storage and implementation notes

QHG is implemented in the QGI reference deployment as a columnar store with a content-addressed immutable layer underneath. The production characteristics are:

- **Node size** — 1,536-dimensional float16 embedding plus a JSON AST; ~4 KB per rule node at rest.
- **Edge size** — type tag, participant hashes, annotation payload; ~256 B per simple hyperedge.
- **Snapshot hash** — Merkle tree over nodes, edges, and type annotations; enables $O(1)$ snapshot equality check.
- **Query latency** — $O(10\text{ ms})$ for node-queries, $O(50\text{ ms})$ for typical neighbourhood-queries with HSC re-ranking, on commodity GPU hardware.
- **Durability** — snapshots are persisted with standard object-storage guarantees; the content-addressed store makes backup and replication straightforward.

The storage layer is not the contribution of this paper; the data *model* is. A QHG can in principle be hosted on any columnar or graph store with the ability to enforce snapshot immutability.

7 Limitations

- **Static edge type taxonomy.** Extensions are possible but must be explicitly versioned. This is a deliberate design choice — avoiding edge-type sprawl — but it is a constraint on domains that need richer edge semantics.
- **No native temporal logic.** Edges can carry time annotations as properties, and the super-session edge captures the primary temporal relation, but QHG does not implement general linear temporal logic. Deployments that need it wrap QHG with an application-level temporal layer.
- **Managed-service realisation.** The reference implementation is a managed deployment with the QAG engine; open-source / local deployments are not supported in v1.0. Teams that require an open substrate can implement QHG on top of a hypergraph-database layer and lose only the managed operational features.

8 Related work

Hypergraphs have a long history in database research and in theoretical computer science [@berge1984hypergraphs]. Applied uses include hypergraph learning [@feng2019hypergraph], recommender systems [@yang2019revisiting], and, recently, retrieval-augmented generation [@luo2024hypergraphrag; @edge2024graphrag]. Immutable data structures and versioning in databases are a standard pattern (Datomic, Dolt, Irmin [@irmin]); their combination with hypergraph semantics is, to our knowledge, new in the agent-memory / regulated AI space.

The Hilbert-space signal layer draws on compositional quantum NLP [@coecke2010discocat; @kartsaklis2021lambeq] and on quantum cognition [@aerts2014quantum; @bruza2015qcognition]. The combination of deterministic parsing, hypergraph state, and signed interference as an edge annotation — the full QHG formulation — is the contribution of the QGI engineering programme, and is the substrate described here.

9 Competing interests

The author is an employee and shareholder of Quantum General Intelligence, Inc., and has a commercial interest in the QAG engine.

10 References

1. [@berge1984hypergraphs] Berge, C. (1984). *Hypergraphs: Combinatorics of Finite Sets.* North-Holland.
2. [@hypergraphdb] Iordanov, B. (2010). *HyperGraphDB: A Generalized Graph Database.* WAIM 2010.
3. [@feng2019hypergraph] Feng, Y., You, H., Zhang, Z., Ji, R., Gao, Y. (2019). *Hypergraph Neural Networks.* AAAI 2019.
4. [@yang2019revisiting] Yang, M. et al. (2019). *Revisiting Hypergraph Neural Networks.*
5. [@edge2024graphrag] Edge, D. et al. (2024). *From Local to Global: A Graph RAG Approach.* arXiv:2404.16130.
6. [@luo2024hypergraphrag] Luo, H. et al. (2025). *HyperGraphRAG.* arXiv:2503.21322.
7. [@irmin] Farr, K. D., Gazagnaire, T. et al. *Irmin: an immutable, distributed database.* OCaml Labs.
8. [@coecke2010discocat] Coecke, B. et al. (2010). *Mathematical Foundations for a Compositional Distributional Model of Meaning.*
9. [@kartsaklis2021lambeq] Kartsaklis, D. et al. (2021). *lambeq.*
10. [@aerts2014quantum] Aerts, D., Sozzo, S. (2014). *Quantum Structure in Cognition.*
11. [@bruza2015qcognition] Bruza, P. D. et al. (2015). *Quantum Cognition.*
12. [@qgi-tr-01] Sammane, S. (2026). *Quantum-Augmented Generation (QAG).* QGI-TR-2026-01.
13. [@qgi-modelcard] Sammane, S. (2026). *Q-Prime public model card.*

11 Cite this as

```
@techreport{sammane2026qhg,
  author      = {Sam Sammane},
  title       = {Quantum HyperGraph (QHG): A First-Class Data Model for Rule-Bearing Knowledg
  institution = {Quantum General Intelligence, Inc.},
  number      = {QGI-TR-2026-05},
```

```
  year        = {2026},
  month       = {April},
  type        = {Preprint},
  note        = {Version 1.0, 21 April 2026. Not peer-reviewed.}
}
```

12 Version history

Version	Date	Notes
v1.0	21 April 2026	Initial preprint release.

© 2025–2026 Quantum General Intelligence, Inc. All rights reserved.